exec.txt
TXT 668.73KB

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Análise Passo a Passo do Arquivo `exec.txt`

1. Identificação do Tipo de Arquivo

O arquivo começa com a assinatura `\x7fELF` (ELF magic number), indicando que é um **arquivo executável Linux** (mais especificamente, um **módulo de kernel Linux**, como veremos).

2. Análise das Strings — Primeira Camada

Logo no início encontram-se strings muito reveladoras:

String	Significado
`diamorphine`	Nome de um **rootkit Linux conhecido publicamente**
`diamorphine_init`	Função de inicialização do módulo
`diamorphine_cleanup`	Função de limpeza (unload) do módulo
`diamorphine_secret`	Segredo usado para comunicação/autenticação
`sys_call_table`	Tabela de chamadas de sistema do kernel
`x64_sys_call`	Ponteiro para syscalls x86_64
`kallsyms_lookup_name`	Função para resolver símbolos do kernel
`/usr/src/linux-headers-7.1.13+parrot7-common/include/linux/ucopysize.h`	Header do kernel usado na compilação
`description=LKM rootkit`	**Metadado explícito: "LKM rootkit"**

String	Significado
`author=m0nad`	Autor conhecido do Diamorphine
`license=Dual BSD/GPL`	Licença do módulo
`name=diamorphine`	Nome do módulo

Conclusão parcial: Trata-se do **Diamorphine**, um rootkit de kernel Linux de código aberto (LKM = Loadable Kernel Module), escrito por "m0nad".

3. Análise das Funções e Símbolos

Os símbolos exportados/definidos no binário incluem:

Símbolo	Função
`hacked_getdents / hacked_getdents64`	Versões **adulteradas** de syscalls de listagem de diretórios
`orig_getdents / orig_getdents64`	Ponteiros para as syscalls originais (para restaurar)
`hacked_kill`	Versão adulterada da syscall `kill`
`orig_kill`	Ponteiro para a syscall original
`give_root`	Função que **concede privilégios de root**
`find_task`	Localiza processos no kernel
`module_hide / module_show`	**Oculta/mostra o módulo** da lista de módulos carregados
`is_invisible`	Verifica se um arquivo/PID deve ser ocultado
`flipswitch_func`	Função de alternância (provavelmente para ativar/desativar o rootkit)
`get_syscall_table_bf`	Brute-force para encontrar `sys_call_table`
`resolve_sym`	Resolve símbolos do kernel

Símbolo	Função
`diamorphine_init / diamorphine_cleanup`	Init/cleanup do módulo
`__sys_call_table`	Referência à tabela de syscalls

Esses são os **componentes clássicos de um rootkit de kernel**.

4. Análise do Comportamento (a partir das strings de debug e código)

4.1 Hook de `getdents` / `getdents64`

O rootkit substitui as syscalls `getdents` e `getdents64` por versões "hacked" que **filtram entradas** dos diretórios. Isso permite:

- Ocultar arquivos (ex.: `diamorphine.ko` , arquivos com prefixo configurado)
- Ocultar processos (PIDs específicos)

4.2 Hook de `kill`

A syscall `kill` é interceptada. Quando um processo envia um sinal mágico (baseado no `diamorphine_secret`), o rootkit:

- Concede root ao processo chamador (`give_root`)
- Ativa/desativa o modo invisível
- Oculta/mostra o módulo

4.3 Ocultação do módulo

`module_hide` remove o módulo da lista `modules` do kernel, tornando-o **invisível** para `lsmod` , `/proc/modules` , etc.

4.4 Brute-force de `sys_call_table`

Como `kallsyms_lookup_name` foi restrito em kernels recentes, o código usa `get_syscall_table_bf` para **localizar a tabela de syscalls por força bruta** — técnica típica de rootkits modernos.

4.5 `give_root`

A função `give_root` chama `prepare_creds` e `commit_creds` para elevar privilégios do processo atual a root.

5. Strings de Debug / Compilação

Presença de:

- `diamorphine.c` — arquivo fonte
- `/home/user/Documents/malwares_rep/Diamorphine` — caminho do repositório onde foi compilado (**"malwares_rep"**)
- Metadados de compilação GCC com flags de kernel
- `.BTF`, `.orc_unwind`, `.retpoline_sites` — seções típicas de módulos de kernel modernos

6. Verificação contra o Projeto Original

O Diamorphine é um projeto público no GitHub (`m0nad/Diamorphine`). As strings, nomes de funções e estrutura batem exatamente com o código-fonte conhecido do rootkit.

7. Classificação de Risco

Característica	Avaliação
Tipo	LKM rootkit (módulo de kernel)
Função	Ocultação de arquivos/processos/módulos + escalada de privilégios
Autor	m0nad (projeto público)
Metadado	`description=LKM rootkit` (auto-declarado)
Técnicas	Hook de syscalls, ocultação de módulo, brute-force de `sys_call_table`, `commit_creds` para root
Alvo	Kernel Linux x86_64 (Parrot OS / headers específicos)

Conclusão Final

Este arquivo é MALICIOSO.

Trata-se do **Diamorphine**, um **rootkit de kernel Linux (LKM)** projetado para:

1. **Ocultar arquivos, processos e o próprio módulo** do sistema
2. **Conceder privilégios de root** a processos que enviem um sinal secreto
3. **Persistir de forma invisível** no kernel

Embora o código seja público e de código aberto (o que permite auditoria), sua **funcionalidade é intrinsecamente maliciosa**: ele subverte o kernel para esconder atividades e escalar privilégios, sendo usado em cenários de pós-exploração, backdoors e persistência de atacantes. A presença de metadados como `description=LKM rootkit` e o caminho `malwares_rep` no binário confirmam sua natureza.

Veredito: MALWARE (rootkit de kernel).

Qual é a categoria principal deste malware?

Categoria Principal do Malware

Classificação

Categoria principal: Rootkit (LKM Rootkit)

Mais especificamente, trata-se de um **Rootkit de Kernel (Kernel-Mode Rootkit)** para Linux, implementado como **LKM (Loadable Kernel Module)**.

Justificativa da Classificação

Evidência	Categoria
Metadado auto-declarado `description=LKM rootkit`	Rootkit
`module_hide` / `module_show` — oculta o módulo da lista do kernel	Rootkit (stealth)

Evidência	Categoria
Hook de `getdents` / `getdents64` — oculta arquivos e processos	Rootkit (stealth)
Hook de `kill` com sinal secreto (`diamorphine_secret`)	Rootkit (backdoor/controle)
`give_root + commit_creds` — escalada de privilégios	Rootkit (privesc)
`sys_call_table + get_syscall_table_bf` — manipulação de syscalls	Rootkit (kernel hooking)
Executa em ring 0 (kernel space)	Rootkit de kernel

Subcategorias Aplicáveis

O Diamorphine também se encaixa em categorias secundárias, mas sua **natureza primária é rootkit**:

1. **Rootkit de Kernel** (principal) — opera no espaço do kernel, máxima furtividade
2. **Backdoor** — concede acesso root via sinal mágico
3. **Ferramenta de Escalada de Privilégios** — `give_root` eleva qualquer processo a root
4. **Ferramenta de Stealth/Persistência** — oculta arquivos, processos e a si mesmo

Distinção Importante

Não é classificado primariamente como:

- **Trojan** — não se disfarça de software legítimo para o usuário final
- **Worm** — não se propaga automaticamente
- **Ransomware** — não criptografa dados
- **Spyware** — não coleta dados por si só (embora possa ser usado como base)

Sua finalidade é **subverter o kernel para ocultar atividades maliciosas e manter acesso privilegiado** — definição clássica de rootkit.

Resposta Direta

Categoria principal: Rootkit de Kernel (Kernel-Mode Rootkit / LKM Rootkit).